

Neural Composer Classifier

Neural Networks and Deep Learning

Team

Tushar Kumar

Viviana Garzon Silva

UNIVERSITY OF SAN DIEGO

SHILEY-MARCOS SCHOOL OF ENGINEERING

Dr. Bilgenur Baloglu

August 8, 2026

Abstract

We developed and compared two deep learning models for a composer identification of the MIDI dataset. In the dataset there were 198 readable MIDI files from Bach, Beethoven, Chopin, and Mozart. Our LSTM used ordered chord tokens, while the CNN used normalized 128×128 piano-roll windows. The LSTM preprocessing produced 25,620 training sequences. CNN preprocessing produced 16,364 base windows and 114,548 augmented training windows using four pitch shifts and two tempo transformations. The CNN test accuracy was 84.36% with a macro F1 = 85.72%, and LSTM was 55.96% accuracy with a macro F1 = 57.03%. These results showed us that the pitch time piano roll representation was better for composer identification. This was because the CNN could learn local pitch and timing from a 2 dimensional piano roll structure, while the LSTM relied on sequential notes.

Introduction

Composer identification requires a model to find musical patterns that are unique to different composers. This project focused on identifying works by Bach, Beethoven, Chopin, and Mozart using two deep learning approaches. We used a LSTM based on note and chord sequences and a CNN based on piano-roll images. We made both models on the same 198-file corpus and evaluated them under the same conditions, allowing us to get a direct comparison between them.

The workflow included data exploration, preprocessing, feature extraction, model construction, training, checkpoint based evaluation, and hyperparameter configuration. Because the files were already MIDI, the preprocessing focused on validation and conversion into numerical representations for deep learning.

MIDI (Musical Instrument Digital Interface) is an event-based digital protocol that captures note pitches, onset times, durations, and velocities but omits acoustic timbre and performance interpretation. This isolates compositional style from variables like performer interpretation and recording quality. Prior work in MIR has shown that both temporal sequence models and image-based models can extract meaningful musical features from MIDI data (Raffel & Ellis, 2014; Oord et al., 2016).

Dataset and Data Exploration

The dataset contained 198 MIDI files, training split contained 166 files and the development/test splits contained 16 files each. Bach and Beethoven contributed 50 files each, Chopin and Mozart had 49 each. All 198 files were pre-divided into train, development and test splits prior to any preprocessing.

Table 1: Dataset Split by Composer

Composer	Train	Dev	Test	Total
Bach	42	4	4	50
Beethoven	42	4	4	50
Chopin	41	4	4	49
Mozart	41	4	4	49
Total	166	16	16	198

The training durations were 280.4 seconds for Bach, 251.7 for Beethoven, 274.8 for Chopin, and 503.4 for Mozart. The overall training set average length was about 5.45 minutes. Mozart files are notable because they had longer lengths which generated more temporal windows.

Data Pre-processing /Feature Extraction

Preprocessing was implemented using the `pretty_midi` library (Raffel & Ellis, 2014) for MIDI parsing. Two independent preprocessing pipelines were constructed, one for the LSTM and one for the CNN. Each pipeline was designed for the specific input format required by the architecture.

For the LSTM pipeline the drum instruments were excluded. The remaining MIDI notes were converted to note names with start times(rounded to 3 digits) and events were sorted by their start time. Notes that began within 0.05 seconds of each other were grouped into one chord token and single notes remained individual tokens. Notes and chords were represented as dot-separated tokens (e.g., C4.E4.G4 for a C major chord). The Mean tempo was computed from tempo-change values and 120 BPM was used when there was no tempo change.

The vocabulary was created using only the training data so information from the development and test sets would not influence the model during training. This led to 34,760 unique note and chord tokens. Each token was converted to an integer index (1-indexed; index 0 reserved for out-of-vocabulary tokens in the LSTM embedding layer) so the LSTM could process the musical sequence numerically. The sequences were divided into windows of 100 tokens with a step size of 10. This made it so the model could learn patterns across short sections instead of processing entire compositions at once. This created 25,620 training windows, 4,249 development windows, and 3,215 test windows.

CNN Piano-Roll Representation

In the CNN the MIDI files were converted into piano-roll matrices because CNNs are designed to recognize spatial patterns. In the piano roll we represented pitch vertically and time was represented horizontally. This gave the CNN, pitch, chord and timing information. Each file was sampled at 10 frames per second and divided into overlapping windows of 128 time frames. The values were then normalized to a range of 0 to 1. In doing this the model received consistent input values during training. Each final CNN input had a shape of (128,128,1) per sample, adding a channel dimension compatible with Conv2D. We had a step size of 32 making overlapping windows so important musical patterns near the edge of one window could also appear in the next. Short files were zero-padded so every input had the same shape. Before the augmentation this gave us 16,364 training windows, 2,340 development windows, and 2,033 test windows.

Data Augmentation

To reduce overfitting and increase training set diversity, the CNN training windows were augmented using two transformations applied to the original 16,364 base windows. We started pitch shifting by ± 1 and ± 2 semitones. Then we implemented by shifting the pitch axis of the piano roll up and down while zeroing out-of-range rows, and tempo scaling by factors of $\times 0.9$ and $\times 1.1$. This was implemented by resampling the time axis and zero-padding back to 128 frames. Each base window produced seven versions including the original. The Training data was increased from 16,364 to 114,548 windows. While doing this we also left the development and test data unchanged.

Preprocessed arrays were saved as `preprocessed/lstm_data.npz` and `preprocessed/cnn_data.npz`, and the token mapping was saved as `vocab.json`. By doing this, model training could be reproduced without repeating feature extraction.

Model Architecture

LSTM Model

The LSTM used a 64-dimensional embedding, a 256-unit LSTM returning sequences, dropout = 0.30, a second 128-unit LSTM, a 64-unit ReLU dense layer, 0.30 dropout layer, and a four-unit softmax output. It used categorical crossentropy and Adam and contained approximately 2.76 million parameters.

CNN Model

The final CNN used two convolutional blocks to learn patterns from the piano-roll inputs. The first Conv2D layer contained 32 filters and the second contained 64 filters, both using 3×3 kernels and ReLU activation. Each convolutional layer was followed by batch normalization and 2×2 max pooling. A dropout rate of 0.25 was applied after the second convolutional block. The model then used GlobalAveragePooling2D (GAP) introduced by Lin et al. (2013), which averaged each feature map to a single value before passing the output to a 128-unit ReLU dense layer. A dropout rate of 0.30 was applied before the final four-unit softmax layer. The model used categorical cross-entropy with the Adam optimizer and contained 28,036 total parameters.

Table 2: LSTM and CNN Configuration

Configuration	LSTM	CNN
Input	100-token sequence	128 x 128x 1 piano roll
Core layers	LSTM 256 LSTM 128	Conv2D 32 Conv2D 64
Dense layer	64	128

Dropout	0.3	0.25 / 0.30
Batch size	256	64
Max epochs	100	50
Early stopping	Patience 10	Patience 8

LSTM and CNN Comparisons

Table 3 presents the overall performance of both models on the development and test sets. The LSTM and CNN processed the MIDI data in extremely different ways. The LSTM learned from sequences of note and chord tokens. The CNN on the other hand, learned from the pitch and timing patterns shown in the piano-roll matrices. The CNN used GlobalAveragePooling2D (GAP) after the convolutional layers to reduce the feature maps before it did its classification. This kept the final CNN relatively small at 28,036 parameters while still allowing it to learn useful musical patterns. The LSTM contained a greater amount of parameters because of its embedding and recurrent layers.

Table 3: Overall Performance Comparison

Split	Metric	LSTM	CNN	Δ (CNN – LSTM)
Dev	Accuracy	57.43%	85.90%	+28.47 pp
Dev	Precision (macro)	55.65%	89.01%	+33.36 pp
Dev	Recall (macro)	61.16%	89.67%	+28.51 pp
Dev	F1 (macro)	55.56%	89.30%	+33.74 pp
Test	Accuracy	55.96%	84.36%	+28.40 pp
Test	Precision (macro)	57.46%	85.61%	+28.15 pp
Test	Recall (macro)	60.93%	86.00%	+25.07 pp
Test	F1 (macro)	57.03%	85.72%	+28.69 pp

Note. pp = percentage points.

Model Training, Optimization, and Evaluation

Training

Random seed 42 was set for NumPy and TensorFlow. The LSTM used batch size 256, a 100-epoch maximum, EarlyStopping on validation accuracy with patience 10, and a best-model checkpoint. Training stopped at epoch 19 and restored epoch 9.

The CNN used batch size 64, a 50-epoch maximum, EarlyStopping with patience 8, and its own best checkpoint. Training stopped at epoch 32 and restored epoch 24, which reached 85.90% development accuracy.

Training and development accuracy and loss curves were plotted for both models to assess convergence and overfitting.

Optimization

An initial CNN version using a Flatten layer after the second convolutional block produced 8.41 million parameters (dominated by the 65,536 x 128 Dense weight matrix) and achieved 79.54% test accuracy with signs of overfitting. According to Goodfellow et al. (2016), pooling is valuable when the goal is to detect whether a feature is present rather than where it is located. We used this logic to replace Flatten with GlobalAveragePooling2D, which compressed the parameters from 8.41M to 28,036 while improving test accuracy by 4.82 percentage points. This improved our model because GAP compresses each feature map into a single average value. This prevents the model from memorizing specific positions in the training windows. It forces the model to learn patterns that work better for the whole piano roll.

Evaluation Procedure and Main Findings

Final evaluation loaded the saved best checkpoints and generated window-level predictions on development and test data. Metrics were accuracy, macro precision, macro recall, and macro F1. Normalized confusion matrices were used to inspect composer-level errors.

Table 4: Final Test-Set Model Comparison

Metric	LSTM	CNN
Accuracy	0.5596	0.8436
Macro precision	0.5746	0.8561
Macro recall	0.6093	0.86
Macro F1	0.5703	0.8572

We found that CNN was the best model for this test. Its Test accuracy was 84.36%, compared with 55.96% for the LSTM, a gain of 28.40 percentage points. The Macro F1 increased from 57.03% to 85.72%. On CNN test windows, Bach achieved F1=1.00 and Chopin F1 = 0.9395; Beethoven and Mozart were more difficult at 0.7455 and 0.7439. The results show that the piano-roll representation generalized significantly better than a 100-token note representation.

Per-Composer CNN Performance

Table 5 presents the CNN's composer classification report on the test set. Bach was classified with perfect accuracy (F1 = 100%), Chopin also performed strongly (F1 = 93.95%). However, Beethoven and Mozart showed the lowest F1 scores (74.55% and 74.39%, respectively), with a substantial portion of misclassifications occurring between these two composers, indicating that the model found the two composers' musical patterns harder to distinguish.

Table 5: CNN Per composer Classification Report - Test Set

Composer	Precision	Recall	F1-Score	Support (windows)
Bach	100.00%	100.00%	100.00%	459
Beethoven	78.14%	71.27%	74.55%	637
Chopin	93.09%	94.84%	93.95%	426
Mozart	71.20%	77.89%	74.39%	511
Accuracy		84.36%		2,033
Macro avg	85.61%	86.00%	85.72%	2,033

Conclusion

From our testing we found the CNN was more effective than the LSTM for identifying composers from MIDI data. The CNN reached 84.36% test accuracy and an 85.72% macro F1-score, while the LSTM reached 55.96% accuracy and had a 57.03% macro F1-score. The biggest difference we found was how differently the models analyzed the music. The LSTM focused on sequences of notes and chords, while the CNN looked at pitch and timing together through piano-roll images. For this dataset, the piano-roll representation gave the model an easier way to recognize patterns and led to better performance.

The results also showed us that using a model that's larger doesn't always lead to a better solution. Our CNN model used much fewer parameters but still generalized better to unseen data. When we replaced Flatten with GlobalAveragePooling2D our parameters got cut by 99.7%. This resulted in us having a much smaller model that still led us to have better results in the end.

Limitations

- The dataset contained 198 MIDI files, which may not represent the full range of each composer's style.
- LSTM was limited by sparse chord vocabulary (34,760 tokens, most seen only once)
- The model classifies individual piano-roll windows rather than the full piece, meaning windows from the same file may predict different composers with no mechanism to predict the correct one
- BatchNormalization affected by the mix of original and augmented windows training, causing unstable validation loss throughout training

Additional Future Work

- Using different audio representations such as chromagrams or constant-Q instead of piano rolls
- Cross validation with a larger dataset to produce more reliable performance estimates
- Gradient-based visualization techniques such as Grad-CAM to identify which time frame influenced the composer prediction

References

- Raffel, C., & Ellis, D. P. W. (2014). Intuitive analysis, creation and manipulation of MIDI data with pretty_midi. In *Proceedings of the 15th International Society for Music Information Retrieval Conference Late Breaking and Demo Papers*.
- M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- McCullum, N. (2020, July 13). *The ultimate guide to recurrent neural networks in Python*. freeCodeCamp. <https://www.freecodecamp.org/news/the-ultimate-guide-to-recurrent-neural-networks-in-python/>
- Lin, M., Chen, Q., & Yan, S. (2013). Network in network. arXiv preprint arXiv:1312.4400. <https://arxiv.org/abs/1312.4400>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <https://www.deeplearningbook.org/>
- TensorFlow. (2024, August 16). Simple audio recognition: Recognizing keywords. https://www.tensorflow.org/tutorials/audio/simple_audio
- Anthropic. (2024). Claude (claude-sonnet-4-6) [Large language model]. <https://www.anthropic.com>

AI Disclosure

Claude (Anthropic, 2024) was used to assist with project workflow planning(project_phases.md), accuracy verification of reported metrics, and APA 7 citation formatting. All technical decisions, model design, training, and analysis were performed by the authors.

Appendix: Github

https://github.com/vgsilva-eng/Neural-Networks/blob/main/neural_composer_classifier.ipynb